Государственное бюджетное общеобразовательное учреждение города Москвы "Школа № 1596"

ПРИЛОЖЕНИЕ ДЛЯ ВЕДЕНИЯ АВТОМОБИЛЬНОГО ЖУРНАЛА

Участники: 10 класс, ГБОУ Школа № 1596, Шорсткин Владислав Михайлович; 10 класс, ГБОУ Школа № 1596, Козлов Никита Алексеевич.
Руководители: лаборант, РТУ МИРЭА, Грачев Александр Витальевич; лаборант, РТУ МИРЭА, Зенина Алиса Александровна.

Москва, 2026

ОГЛАВЛЕНИЕ

1. ВВЕДЕНИЕ3
Актуальность работы3
Обоснование выбора темы3
Постановка проблемы3
2. ЦЕЛЬ И ЗАДАЧИ РАБОТЫ4
Цель4
Задачи работы4
3. МЕТОДИКА ВЫПОЛНЕНИЯ РАБОТЫ4
Этапы и сроки выполнения работы4
Принципы проектирования и выбор технологий5
Проектирование архитектуры5
Разработка ключевых алгоритмов7
Среда разработки и оборудование10
4. РЕЗУЛЬТАТЫ РАБОТЫ И ИХ ПРОВЕРКА10
Созданный продукт10
Методы и сценарии тестирования12
Результаты тестирования и апробация12
5. ОПИСАНИЕ ПОЛУЧЕННОГО ПРОДУКТА И ПЕРСПЕКТИВЫ РАБОТЫ13
Назначение и ключевые возможности приложения13
Интерфейс и удобство использования14
Практическая значимость и внедрение21
Перспективы развития проекта22
Заключение23
6. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ25
7. ПРИЛОЖЕНИЕ А27

1 ВВЕДЕНИЕ

Актуальность работы

Для современных владельцев автомобилей актуальной остаётся проблема систематизации технического обслуживания (ТО). Отсутствие простого и наглядного инструмента для учёта выполненных работ и планирования будущих часто приводит к пропуску регламентных процедур, что влечёт за собой повышенный износ агрегатов, снижение безопасности и неоправданные расходы на ремонт. Существующие решения, как правило, либо являются перегруженными функциями сложными сервисами, либо не позволяют гибко адаптироваться под конкретную модель автомобиля и стиль вождения пользователя. Наше приложение решает эту проблему и представляет собой простую локальную систему учёта, не требующую от пользователя затрат на понимание интерфейса, а также позволяет гибко настроить необходимые процедуры планового ТО для конкретного автомобиля.

Обоснование выбора темы

В ходе наблюдений за владельцами автомобилей стало понятно, что большинство из них не имеет системы учёта или даже записей или заметки для отслеживания износа запчастей автомобиля и проведения ТО. Так же многие забывают все плановые работы из руководства по эксплуатации автомобиля и далее чинят те части автомобиля, которые сломались, не заботясь о предупреждении этих поломок. Пренебрежение к собственному имуществу и безопасности побудило написать простое приложение, которое позволяло бы просто следить за автомобилем и не перегружать пользователя дополнительным функционалом.

Постановка проблемы

Таким образом, можно констатировать отсутствие на рынке бесплатного, минималистичного desktop-приложения, которое позволяло бы вести персональный журнал ТО, гибко настраивать регламент процедур для

конкретного автомобиля и автоматически определять необходимость ближайшего обслуживания на основе текущего пробега, предоставляя пользователю интуитивно понятный интерфейс.

2 ЦЕЛЬ И ЗАДАЧИ РАБОТЫ

Цель

Разработать минималистичное и удобное ПО для ведения журнала обслуживания автомобиля и определения необходимого ТО в зависимости от пробега автомобиля.

Задачи работы

1. Разработать структурную схему ПО;
2. Написать код, реализующий:
    a) графический интерфейс приложения;
    b) логику взаимодействия с информацией;
    c) настройку процедур обслуживания для различных машин;
3. Составить описание проекта и презентацию для защиты.

3 МЕТОДИКА ВЫПОЛНЕНИЯ РАБОТЫ

Этапы и сроки выполнения работы

Работа выполнялась в течение 2025-2026 учебного года на базе ГБОУ школы №1596 в рамках проекта «Приложение для ведения автомобильного журнала», в три основных этапа в период с 01.12.2025 по 18.01.2026:

1. Этап проектирования и анализа 01.12.2025 — на этом этапе были сформулированы принципы разработки, проанализированы аналоги, спроектирована архитектура и алгоритмы.

2. Этап программной реализации 14.12.2025 — включал написание кода модулей, разработку графического интерфейса и их интеграцию.

3. Этап тестирования и отладки 12.01.2026 — был посвящён проверке функционала, исправлению ошибок и подготовке документации.

Принципы проектирования и выбор технологий

Для проектирования приложения, которое решает поставленную проблему были сформулированы опорные принципы, которые определили конечные вид и работу приложения.

1. Минимализм. В соответствии с целью проекта основным принципом был выбран минимализм. Это выразилось в отказе от лишних функций как подсчёт затраченных средств на обслуживание автомобиля, штрафы и аварии, указание стоимости запчастей и т.д., так же в простоте архитектуры программы, для простоты поддержки и подработки.

2. Гибкость - второй важнейший принцип. Гибкость приложения заключается в возможности настроить под себя любые процедуры планового ТО и внести так же внеплановые процедуры в историю обслуживания. Так же можно настроить интервалы и допуски для перехода статусов плановых процедур ТО, которые отражают скорость износа частей автомобиля и предварительное напоминание о необходимости скорого проведения обслуживания.

Для реализации программы в соответствии с принципами в был подобран следующий стек технологий:

- Язык программирования Python 3.10 благодаря его кроссплатформенности, простоте синтаксиса и богатой экосистеме библиотек.
- Для создания графического интерфейса - стандартная библиотека Tkinter, что гарантирует стабильность работы, отсутствие внешних зависимостей и простоту развёртывания приложения на любом ПК пользователя.
- Для хранения данных - встроенная в Python СУБД SQLite, которая не требует отдельной установки и подходит для локальных однопользовательских приложений, обеспечивая надёжное и быстрое взаимодействие с данными.

Проектирование архитектуры

На основе сформулированных принципов и анализа необходимых пользовательских функций, отражённых на диаграмме вариантов использования на рисунке 1, была спроектирована модульная архитектура.

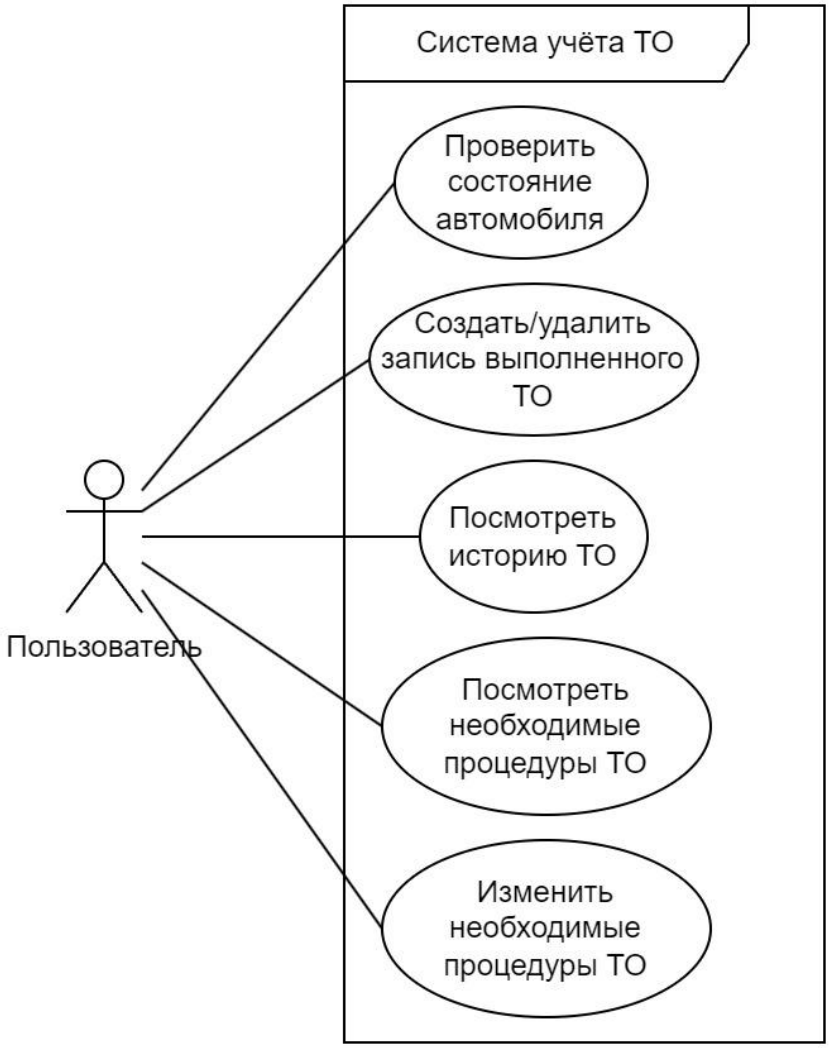


Рисунок 1 - Диаграмма вариантов использования.

Архитектура, представленная на рисунке 2, разделяет функционал между независимыми модулями:

- «main.py» является главным исполняемым скриптом. В нём содержится разметка графического интерфейса, а также процедура запуска всего приложения.
- «config.py» изменяет пользовательские настройки (список процедур ТО и интервалы их повторения, «допуск»), которые сохраняются в файле «car_config.json».
- «database.py» является API для взаимодействия с локальной базой данных «car_logger.db». Модуль осуществляет подключение, создание, редактирование базы данных, предоставляя верхнеуровневые функции.

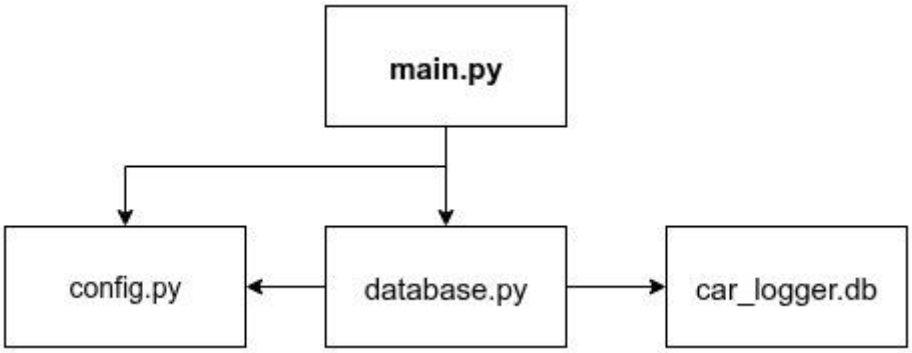


Рисунок 2 - Структурная диаграмма ПО.

Такое разделение упростило параллельную разработку, тестирование и отладку каждого компонента системы.

Разработка ключевых алгоритмов

Основной функцией этого приложения является же определение того какие процедуры планового ТО нужно провести в ближайшее время на основе пробега автомобиля и учёт уже проведённых процедур.

Главные функции приложения обеспечили то, что центральным алгоритмом приложения является алгоритм определения статуса процедуры ТО на основе пробега. Его логика учитывает не только факт превышения интервала проведения процедуры ТО, но и вводит понятие «допуска» — процента от интервала, за который система предупреждает о скором ТО. Это позволяет пользователю гибко адаптировать напоминания под свой стиль вождения и образ жизни.

Алгоритм определения необходимости ТО, блок-схема которого представлена на рисунке 3, реализован таким образом, что каждая из процедур может быть в одном из следующих состояний:

- «В норме». Необходимость в данной процедуре отсутствует.
- «Скоро потребуется обслуживание». Это состояние означает близость данной процедуры ТО, что даёт возможность владельцу грамотно спланировать данную процедуру в ближайшее время.
- «Необходимо обслуживание». Необходимость в данной процедуре присутствует.

Данный алгоритм работает для каждой из процедур со своим параметром «допуска». Этот параметр позволяет определить за сколько километров (в процентах от периода обслуживания) необходимо сменить статус процедуры на «Скоро потребуется обслуживание».

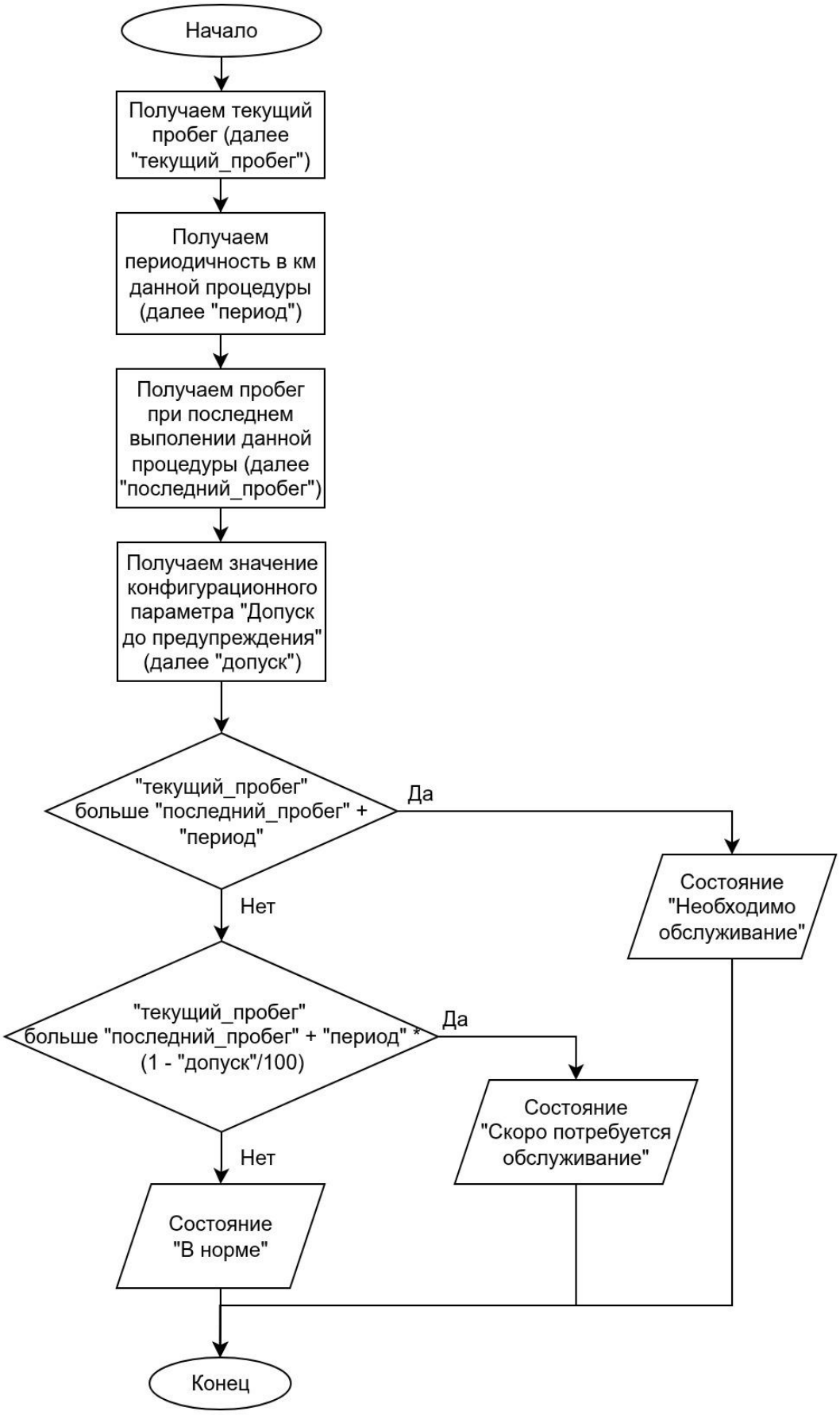


Рисунок 3 - Блок-схема алгоритма для определения статуса процедуры ТО.

На рисунке 4 изображена иллюстрация состояний процедуры ТО в зависимости от пробега. Переменная «период» для каждой процедуры своя и объявляется в перечне необходимого ТО. Переменная «допуск» является настройкой, которую можно изменить. К примеру, автолюбителю в городских условиях допуск можно уменьшить в следствие небольших поездок и большого количества сервисов поблизости, а в сельских условиях - увеличить из-за больших расстояний и меньшего количества СТО.

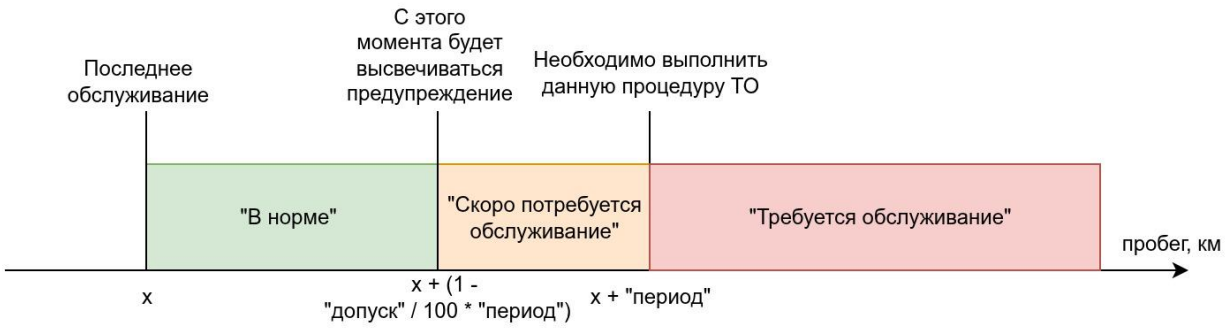


Рисунок 4 - Иллюстрация состояний процедуры ТО в зависимости от пробега.

Среда разработки и оборудование

Разработка велась на ПК под управлением Windows 10 в среде программирования VS Code. Для контроля версий использовался Git, исходный код размещён на GitHub.

Для формирования исходного перечня плановых работ и их интервалов в качестве примера было использовано руководство по эксплуатации автомобиля Toyota RAV4 2016 года выпуска.

4 РЕЗУЛЬТАТЫ РАБОТЫ И ИХ ПРОВЕРКА

Созданный продукт

В результате выполнения поставленных задач было разработано desktop-приложение «Приложение для ведения автомобильного журнала» на языке программирования Python. Приложение представляет собой программный комплекс, предназначенный для ведения электронного журнала сервисных

работ и автоматического контроля сроков планового обслуживания на основе текущего пробега транспортного средства.

Ключевые характеристики приложения:

1. Графический пользовательский интерфейс (GUI) – на основе библиотеки Tkinter создан интуитивно понятный интерфейс, включающий главное меню, диалоговые окна для ввода данных, табличное представление информации и цветовую индикацию статусов.

2. Локальная база данных – для хранения истории обслуживания реализовано взаимодействие с SQLite через модуль «database.py». База данных обеспечивает надёжное хранение записей (пробег, дата, тип работы, описание) и предоставляет методы для добавления, выборки, удаления записей и определения последнего выполненного сервиса.

3. Интеллектуальный алгоритм определения статуса ТО – в ядре системы реализован механизм автоматического сравнения текущего пробега с запланированными интервалами обслуживания для каждой процедуры. Алгоритм учитывает настраиваемый процент допуска, что позволяет формировать два уровня предупреждений: «Скоро потребуется» и «СРОЧНО!».

4. Модульная архитектура – приложение построено по принципу разделения ответственности: логика данных («database.py»), настройки («config.py») и интерфейса («main.py»).

5. Реализованы все запланированные функции:

1) Ведение журнала сервисных записей с ручным и плановым вводом.
2) Просмотр истории с возможностью удаления записей.
3) Проверка текущего состояния автомобиля с детализированным отчётом.
4) Полная настройка списка плановых работ и интервалов обслуживания.
5) Сохранение и загрузка пользовательских настроек.

Таким образом создано минималистичное и гибко настраиваемое программное обеспечение, которое позволяет пользователю систематизировать учёт технического обслуживания автомобиля и получать своевременные

уведомления о приближающихся сервисных работах на основе актуального пробега.

Методы и сценарии тестирования

Для тестирования реализованного приложения были использованы следующие методы:

1. Модульное тестирование: проверка каждого компонента отдельно (работа «database.py», чтение/запись «config.py», отклик интерфейса).
2. Функциональное тестирование: проверка основных пользовательских сценариев по диаграмме вариантов использования (добавление/удаление записи, проверка состояния, настройка ТО, просмотр истории).
3. Проверка корректности алгоритма: проверка логики определения статуса («В норме», «Скоро потребуется обслуживание», «Требуется обслуживание») на специально подобранных контрольных точках пробега.
4. Тестирование на ошибочные данные: проверка устойчивости интерфейса к вводу букв вместо цифр, отрицательных значений и т.д.

Результаты тестирования и апробация

Приложение было протестировано по обозначенной методологии, чтобы проверить его надёжность и соответствие цели.

Модульное тестирование продемонстрировало стабильную работу всех компонентов. База данных стабильно выполняет запросы на добавление, поиск и удаление записей. Настройки корректно загружаются и сохраняются. Графический интерфейс реагирует на действия пользователя без задержек.

Функциональные тесты подтвердили, что основные возможности приложения работают, как задумано. Можно добавлять записи о ремонтах и плановом обслуживании, проверять состояние автомобиля по пробегу, просматривать всю историю и настраивать список необходимых работ. В ходе апробации приложения среди потенциальных пользователей (преподаватели, родители) было отмечено, что выполнить перечисленные выше операции не сложно и интуитивно понятно каким образом нужно взаимодействовать с ПО.

Особое внимание было уделено проверке алгоритма, который определяет, когда нужно выполнить следующее ТО. На 15 контрольных значениях пробега система каждый раз правильно выдавала один из трёх статусов: «В норме», «Скоро потребуется обслуживание» или «Требуется обслуживание». Расчёты учитывают заданные интервалы и процент допуска.

Приложение устойчиво к ошибкам ввода. Если пользователь вводит буквы вместо цифр в поле пробега, отрицательные значения или неправильную дату, программа обрабатывает исключительные ситуации и выводит понятное сообщение об ошибке.

Комплексное тестирование и апробация подтвердили, что разработанное приложение полностью соответствует поставленной цели и решает сформулированную проблему. Все её части работают согласованно, интерфейс удобен, а алгоритмы выполняют свои задачи точно и без сбоев.

5 ОПИСАНИЕ ПОЛУЧЕННОГО ПРОДУКТА И ПЕРСПЕКТИВЫ РАБОТЫ

Назначение и ключевые возможности приложения

Разработанный продукт «Приложение для ведения автомобильного журнала» представляет собой готовый к использованию программный комплекс, который позволяет автовладельцу эффективно и без лишних усилий управлять техническим обслуживанием своего автомобиля.

Основная цель продукта — избавить пользователя от необходимости вести бумажные записи, запоминать сроки сервиса и вручную отслеживать пробег для каждой процедуры. Программа автоматизирует эти рутинные задачи, предоставляя следующие ключевые возможности:

- Централизованное хранение истории. Вся информация о работах, датах и пробеге сохраняется в локальной базе данных. Это надёжно защищает данные от потери и позволяет мгновенно получить доступ к полной истории обслуживания.
- Автоматический контроль сроков ТО. Система анализирует текущий пробег и сравнивает его с заданными интервалами для каждого типа работ. Приложение

само определяет, когда обслуживание нужно в ближайшее время или срочно, и наглядно информирует пользователя, помогая планировать визиты на сервис.

- Гибкая адаптация под автомобиль. Пользователь может настроить перечень процедур, изменить интервалы пробега и порог предупреждений. Это делает приложение универсальным для любой марки машины и условий эксплуатации.
- Простой и понятный интерфейс. Основные действия выполняются в 1–2 клика. Интерфейс спроектирован интуитивно и не требует специального обучения, что делает программу доступной для пользователей с любым уровнем подготовки.

Интерфейс и удобство использования

Принцип минимализма и удобства, заложенный в основу проектирования, реализован в интуитивно понятном графическом интерфейсе (GUI). Основные функции доступны с главного окна, изображённого на рисунке 5. Для демонстрации работоспособности и логики интерфейса ниже представлены ключевые экраны приложения.

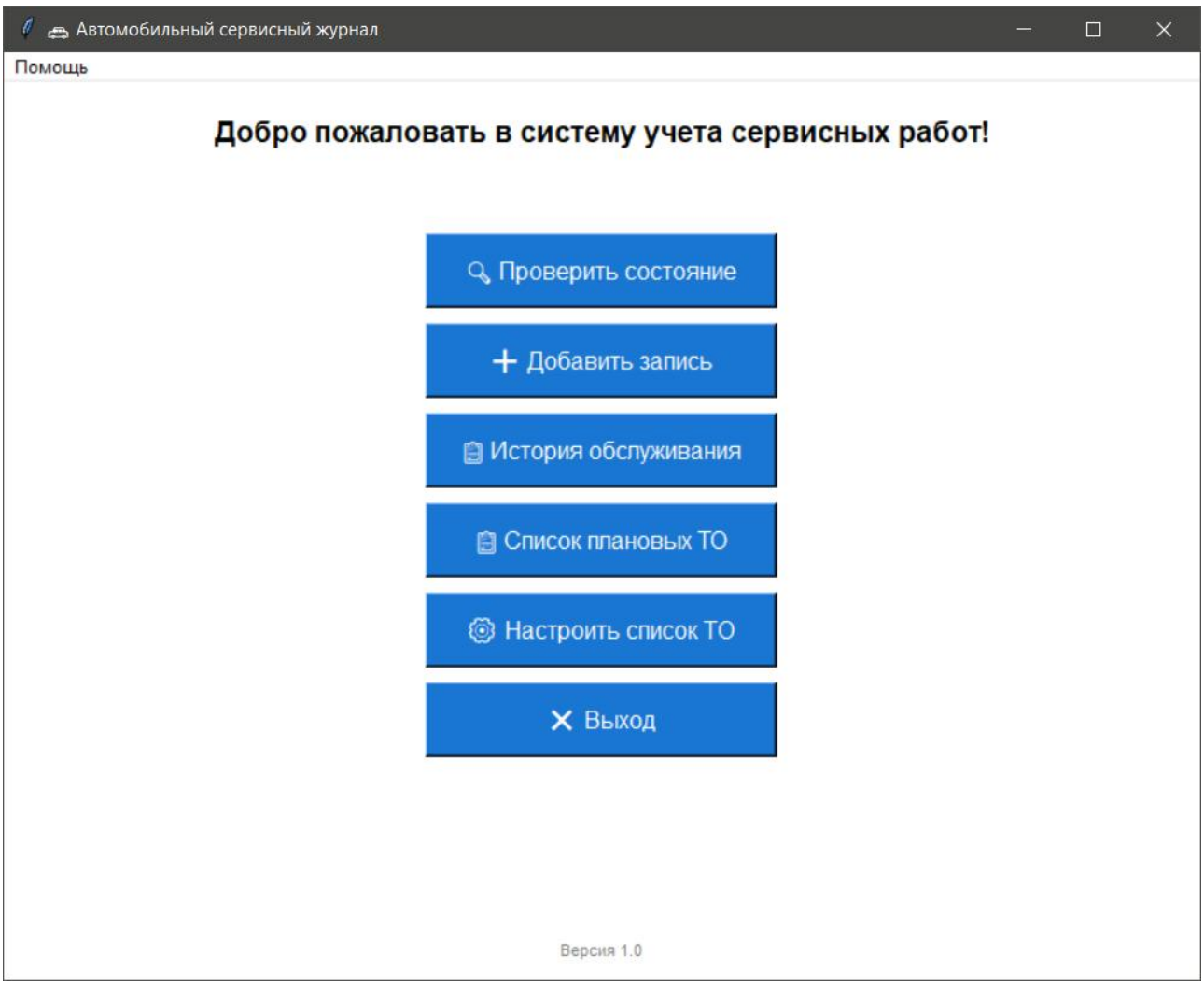


Рисунок 5 - Главное окно программы.

Рисунке 6 изображено окно проверки состояния. Все процедуры в норме, интерфейс использует зелёную цветовую палитру.

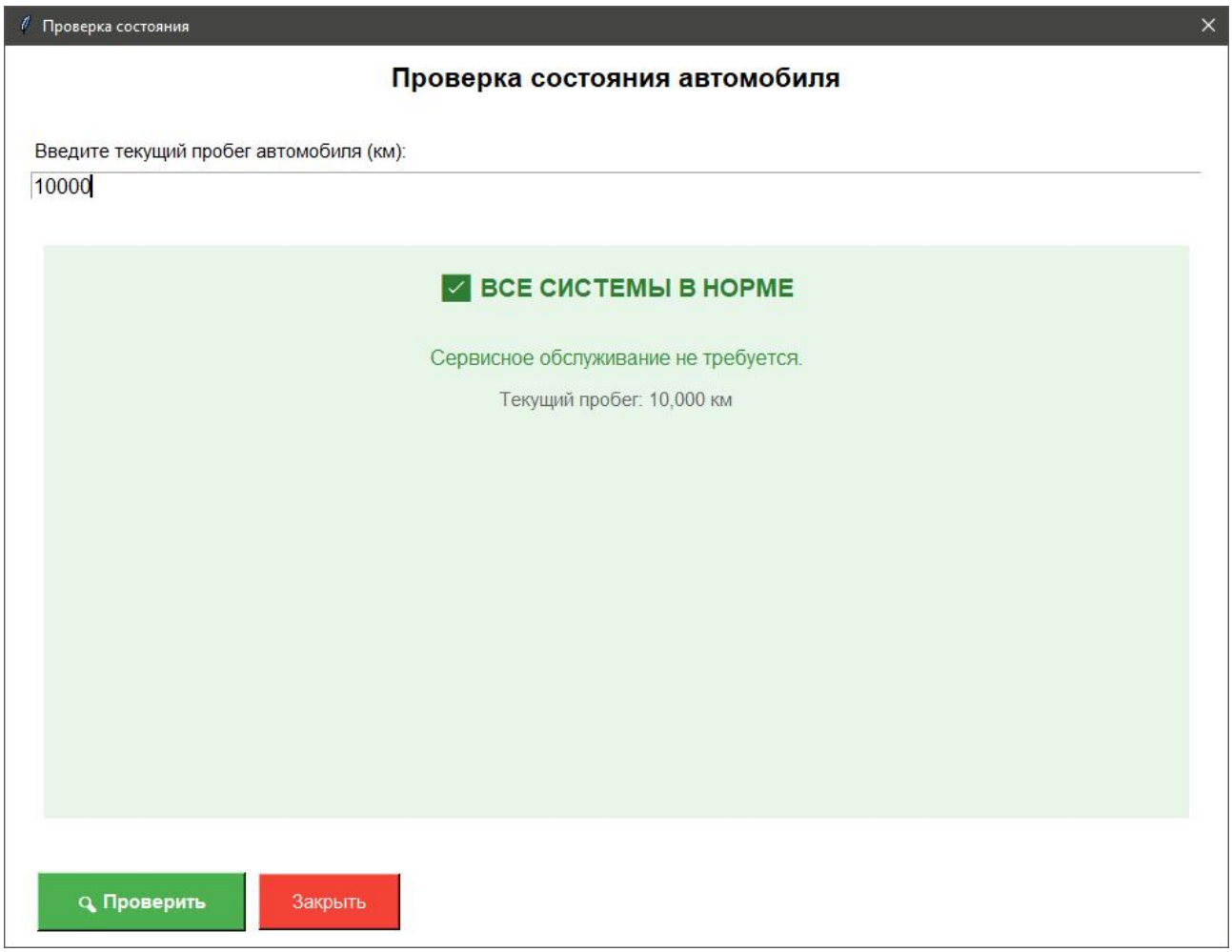


Рисунок 6 - Проверка состояния автомобиля. Всё в норме.

На рисунке 7 изображено окно проверки состояния. Система выявила процедуры, требующие скорого выполнения, выделив их оранжевым цветом.

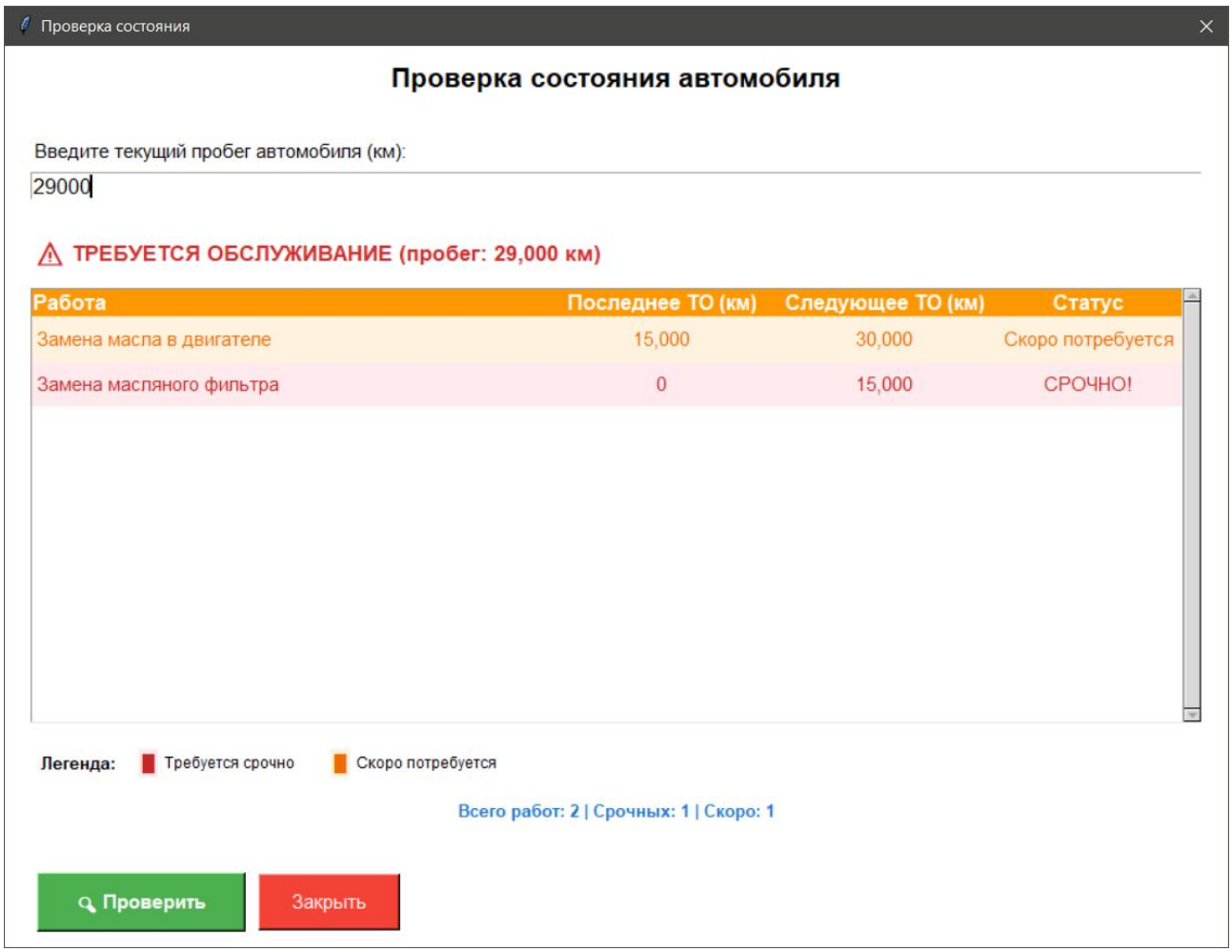


Рисунок 7 - Проверка состояния автомобиля. Скоро потребуется обслуживание.

На рисунке 8 изображено окно проверки состояния. Система выявила процедуры, требующие срочного обслуживания, выделив их красным цветом.

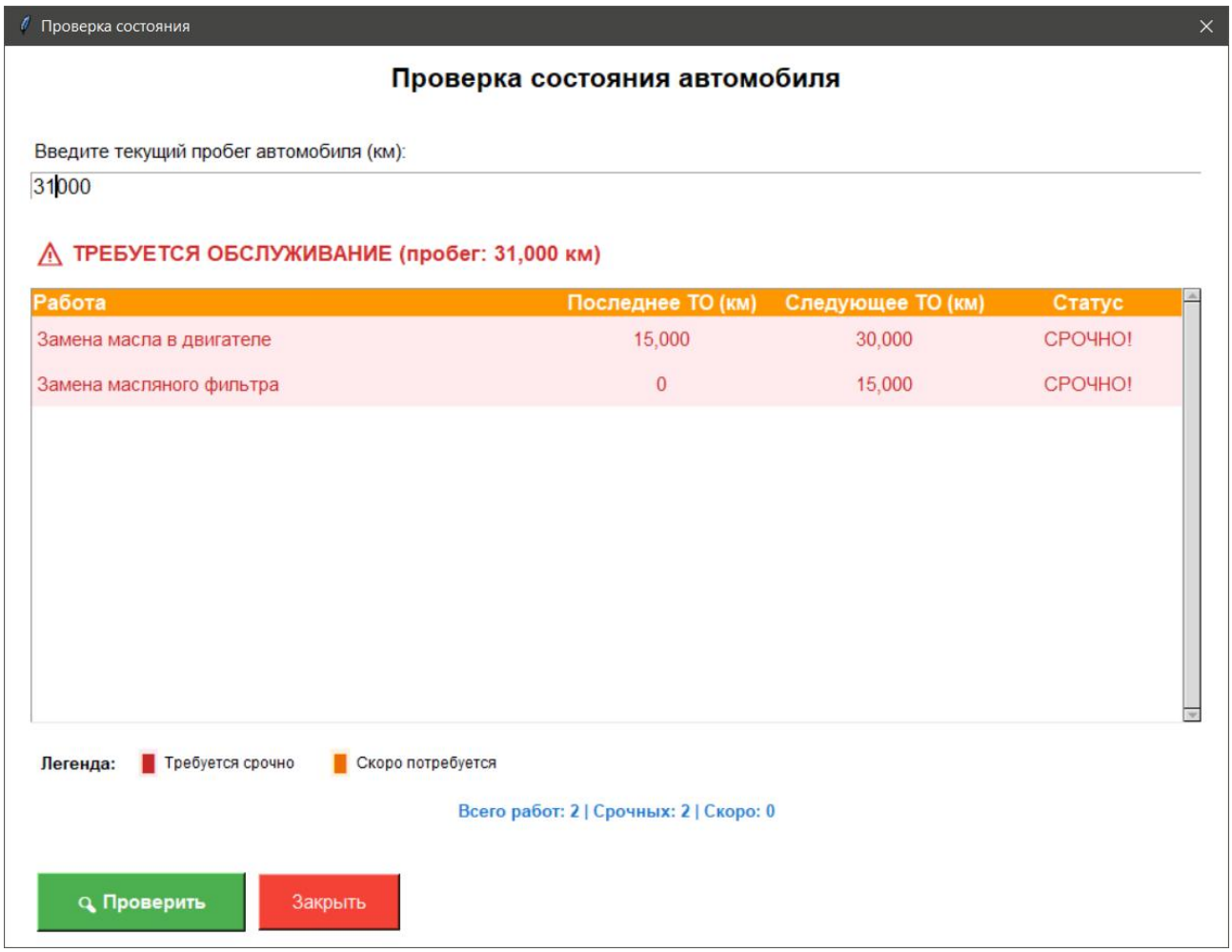


Рисунок 8 - Проверка состояния автомобиля. Требуется обслуживание.

На рисунке 9 изображено окно просмотра истории обслуживания. Записи можно удалить из истории, нажав на кнопку «Удалить» в строке записи.

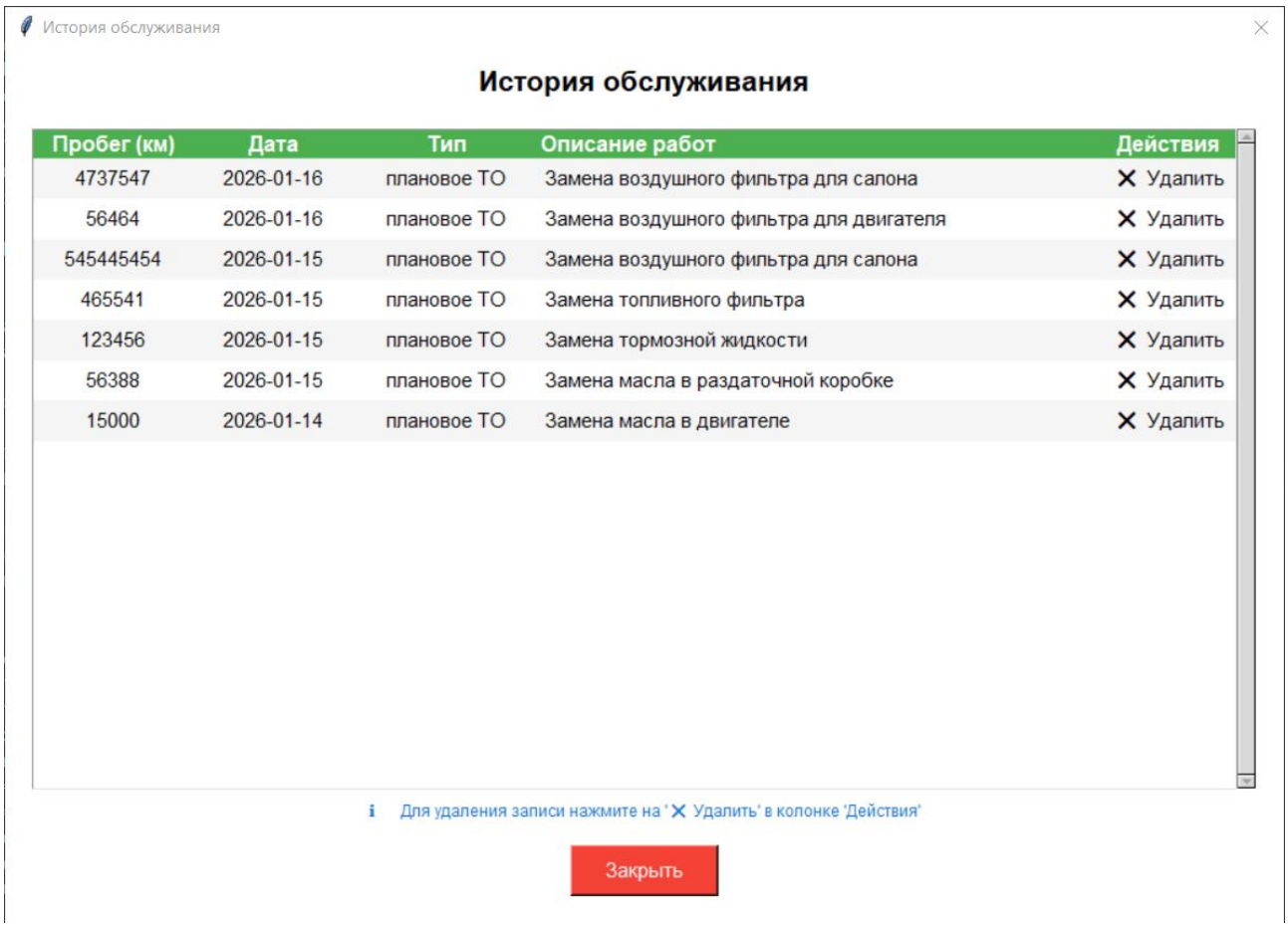
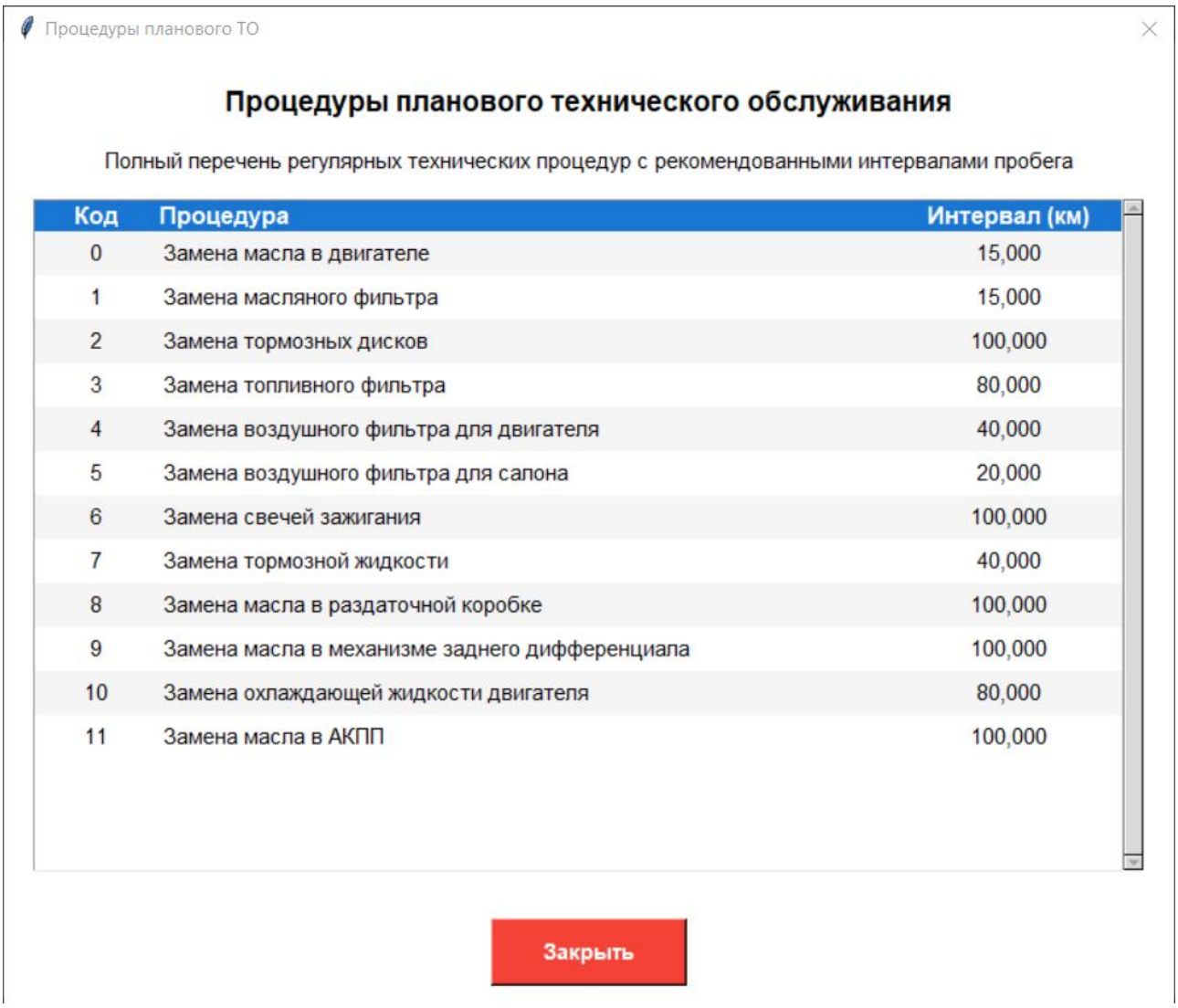


Рисунок 9 - История обслуживания

На рисунке 10 изображено окно существующего списка плановых процедур ТО, которые отслеживаются системой и интервал для их выполнения в км.



Код	Процедура	Интервал (км)
0	Замена масла в двигателе	15,000
1	Замена масляного фильтра	15,000
2	Замена тормозных дисков	100,000
3	Замена топливного фильтра	80,000
4	Замена воздушного фильтра для двигателя	40,000
5	Замена воздушного фильтра для салона	20,000
6	Замена свечей зажигания	100,000
7	Замена тормозной жидкости	40,000
8	Замена масла в раздаточной коробке	100,000
9	Замена масла в механизме заднего дифференциала	100,000
10	Замена охлаждающей жидкости двигателя	80,000
11	Замена масла в АКПП	100,000

Рисунок 10 - Процедуры планового ТО

На рисунке 11 изображено окно настройки процедур ТО. Здесь можно внести процедуру в список отслеживаемых или удалить, а так же отредактировать параметр «допуск». Механика сохранения реализована так, чтобы пользователь не подтверждал каждое действие, а в конце работы сохранял созданный им список сразу целиком.

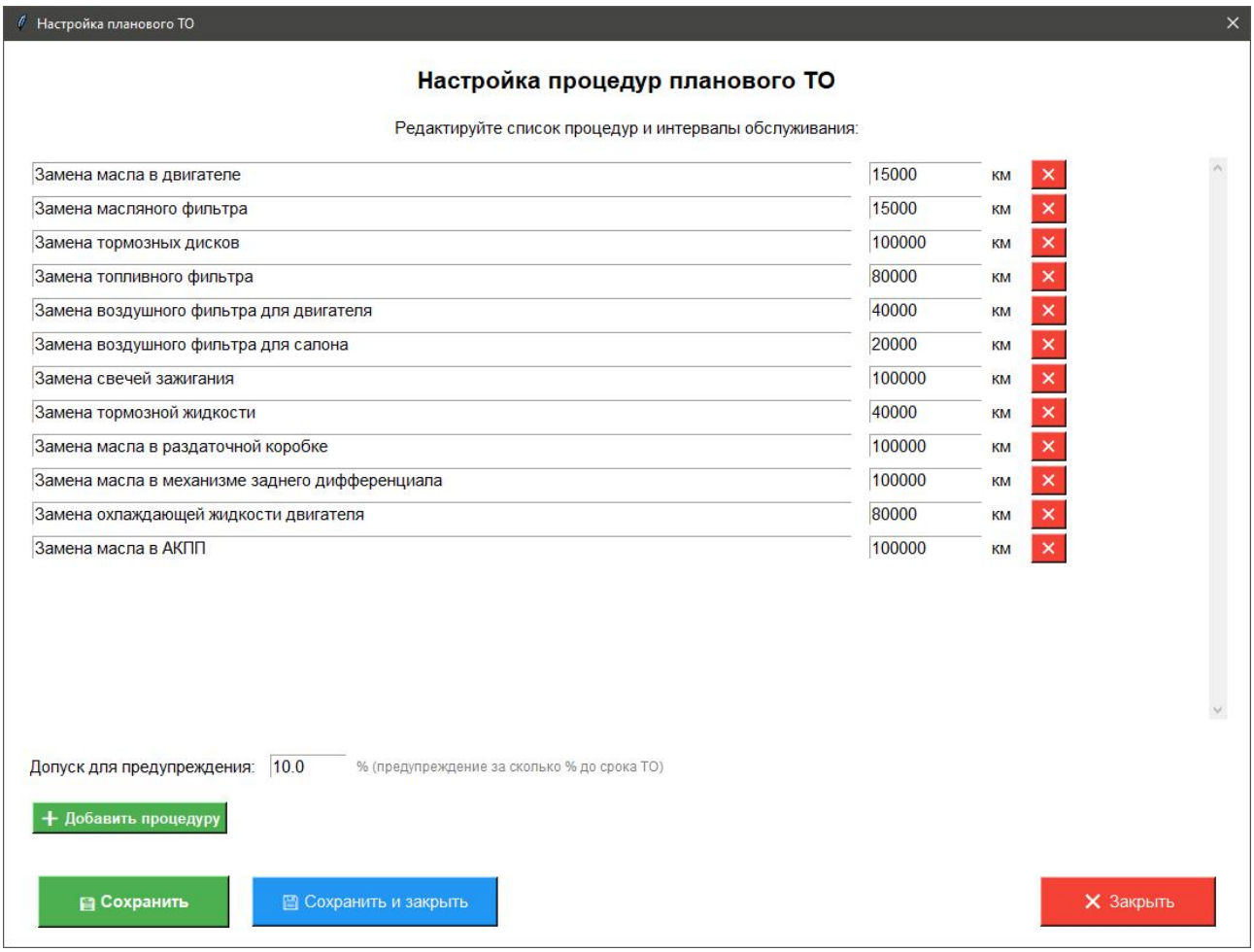


Рисунок 11 - Настройка планового ТО

Практическая значимость и внедрение

Разработанное приложение обладает практической значимостью для широкого круга автовладельцев. Оно позволяет систематизировать разрозненные данные об обслуживании автомобиля, перейти от реактивной модели ремонта («по факту поломки») к превентивному плановому обслуживанию. Это способствует повышению безопасности эксплуатации, снижению риска внезапных отказов и оптимизации расходов на содержание транспортного средства за счёт своевременного проведения необходимых работ.

Технически приложение готово к использованию, инструкция по его развёртке находится в приложении А. Развёртка не является технически сложной и в будущем может быть упрощена до пары кликов.

В ходе демонстрации приложение оно было опробовано представителями целевой аудитории, которые подтвердили, что интерфейс интуитивно понятен и решаемая задача полезна и актуальна.

Благодаря использованию кроссплатформенного языка программирования Python и отсутствию требований к интернет-соединению, приложение может быть легко распространено среди автовладельцев через публичные репозитории (Github).

Исходя из этого, можно сказать что приложение переходит из категории учебного в категорию пользовательского и готово к практическому применению.

Перспективы развития проекта

Модульная архитектура и выбор масштабируемых технологий (SQLite, Python) закладывают прочный фундамент для дальнейшего развития проекта. Развитие можно вести по следующим основным направлениям:

1. Расширение функциональности для пользователя:

- Поддержка нескольких автомобилей в рамках одного экземпляра приложения с переключателем между профилями.
- Детальная аналитика и визуализация: отображений всех процедур ТО в виде сортированного списка, при проверке состояния машины, с указанием количества километров пробега до проведения ТО, прогрессбары для визуализации оставшегося срока до проведения ТО, инфографика для истории обслуживания.
- Учёт временного фактора: интеграция расчёта необходимости ТО не только по пробегу, но и по времени (например, «раз в год или каждые 15 000 км»), как это рекомендовано во многих руководствах по эксплуатации. Тогда же можно включить отслеживание оставшегося срока страховки.

2. Технологическая интеграция и автоматизация:

- Синхронизация с внешними устройствами: Реализация возможности импорта текущего пробега через OBD-II адаптер или вручную из мобильного приложения автомобиля.
- Облачная синхронизация: Создание легковесного облачного бэкенда для резервного копирования данных и синхронизации истории между несколькими устройствами пользователя (ПК, смартфон).

3. Создание мобильной версии: разработка мобильной версии на том же логическом ядре (алгоритмы, работа с БД), но с интерфейсом, адаптированным для смартфонов. Добавление напоминаний о необходимости проведения ТО.

Реализация данных перспектив не требует переписывания основной логики приложения, доказывая, что технологии и архитектура приложения были выбраны верно.

Заключение

В рамках данной работы была успешно решена актуальная проблема отсутствия на рынке простого, гибкого и бесплатного инструмента для систематического учёта технического обслуживания автомобилей.

Поставленная цель достигнута: разработано минималистичное, удобное и кроссплатформенное desktop-приложение «Приложение для ведения автомобильного журнала» для ПК. В процессе работы выполнены все поставленные задачи: спроектирована модульная архитектура, реализован графический интерфейс, создано ядро в виде алгоритма интеллектуального определения статуса ТО и обеспечено взаимодействие с локальной базой данных.

Практическая значимость проекта подтверждена комплексным тестированием и апробацией, которые доказали корректность работы всех компонентов, устойчивость к ошибкам и удобство интерфейса для конечного пользователя.

Разработанный продукт представляет собой законченное, работоспособное решение, готовое к практическому использованию.

Выбранные технологические решения и модульная архитектура открывают широкие перспективы для дальнейшего развития проекта.

6 СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. Алиса Главная / Авто / Какие приложения позволяют вести учет технического обслуживания автомобилей? / Алиса [Электронный ресурс] // Вопросы к Поиску с Алисой : [сайт]. — URL: https://ya.ru/neurum/c/avto/q/kakie_prilozheniya_pozvolyayut_vesti_uchet_aa5d3090 (дата обращения: 03.12.2025).

2. Какие приложения позволяют вести учет технического обслуживания автомобилей? / [Электронный ресурс] // yandex.ru : [сайт]. — URL: https://yandex.ru/q/question/kakie_prilozheniia_pozvoliaiut_vesti_d8ba008e/ (дата обращения: 03.12.2025).

3. cyberbanana777 school_car_logger / cyberbanana777 [Электронный ресурс] // Github : [сайт]. — URL: https://github.com/cyberbanana777/school_car_logger (дата обращения: 18.01.2026).

4. Python Software Foundation tkinter — Python interface to Tcl/Tk / Python Software Foundation [Электронный ресурс] // docs.python.org : [сайт]. — URL: https://docs.python.org/3.10/library/tkinter.html (дата обращения: 18.01.2026).

5. Python Software Foundation Python 3.10.19 documentation / Python Software Foundation [Электронный ресурс] // docs.python.org : [сайт]. — URL: https://docs.python.org/3.10/index.html (дата обращения: 18.01.2026).

6. Python Software Foundation sqlite3 — DB-API 2.0 interface for SQLite databases / Python Software Foundation [Электронный ресурс] // docs.python.org : [сайт]. — URL: https://docs.python.org/3.10/library/sqlite3.html (дата обращения: 18.01.2026).

7. Python SQLite - CRUD Operations / [Электронный ресурс] // geeksforgeeks.org : [сайт]. — URL: https://www.geeksforgeeks.org/python/python-sqlite-crud-operations/ (дата обращения: 18.01.2026).

8. Python Tkinter CRUD application with SQLite / [Электронный ресурс] // w3resource.com : [сайт]. — URL: https://www.w3resource.com/python-exercises/tkinter/python-tkinter-file-operations-and-integration-exercise-12.php (дата обращения: 18.01.2026).

9. Введение в SQLite Python / [Электронный ресурс] // pythonru.com : [сайт]. — URL: https://pythonru.com/biblioteki/vvedenie-v-sqlite-python (дата обращения: 18.01.2026).
10. Установка интерпретатора и настройка среды разработки: первый шаг в программировании / [Электронный ресурс] // education.yandex.ru : [сайт]. — URL: https://education.yandex.ru/handbook/python/article/interpretatora-i-nastroika-sredi-razrabotki-pervii-shag-v-programmirovanii (дата обращения: 18.01.2026).

7 ПРИЛОЖЕНИЕ А

Мы предполагаем, что используется компьютер с операционной системой Windows 10 и на него установлен интерпретатор python3 версии 3.10. Инструкцию по установке можно прочитать в интернете (см. Список используемой литературы [10]) Чтобы установить приложение, необходимо последовательно сделать следующие шаги:

1. Скачиваем репозиторий с Github[3] в виде zip-архива;
2. Распаковываем архив в удобное место;
3. Открываем интерфейс командной строки в приложении «Проводник», находясь в папке проекта, нажав в любое свободное место окна правой кнопкой мыши. Как это сделать изображено на рисунке 12.

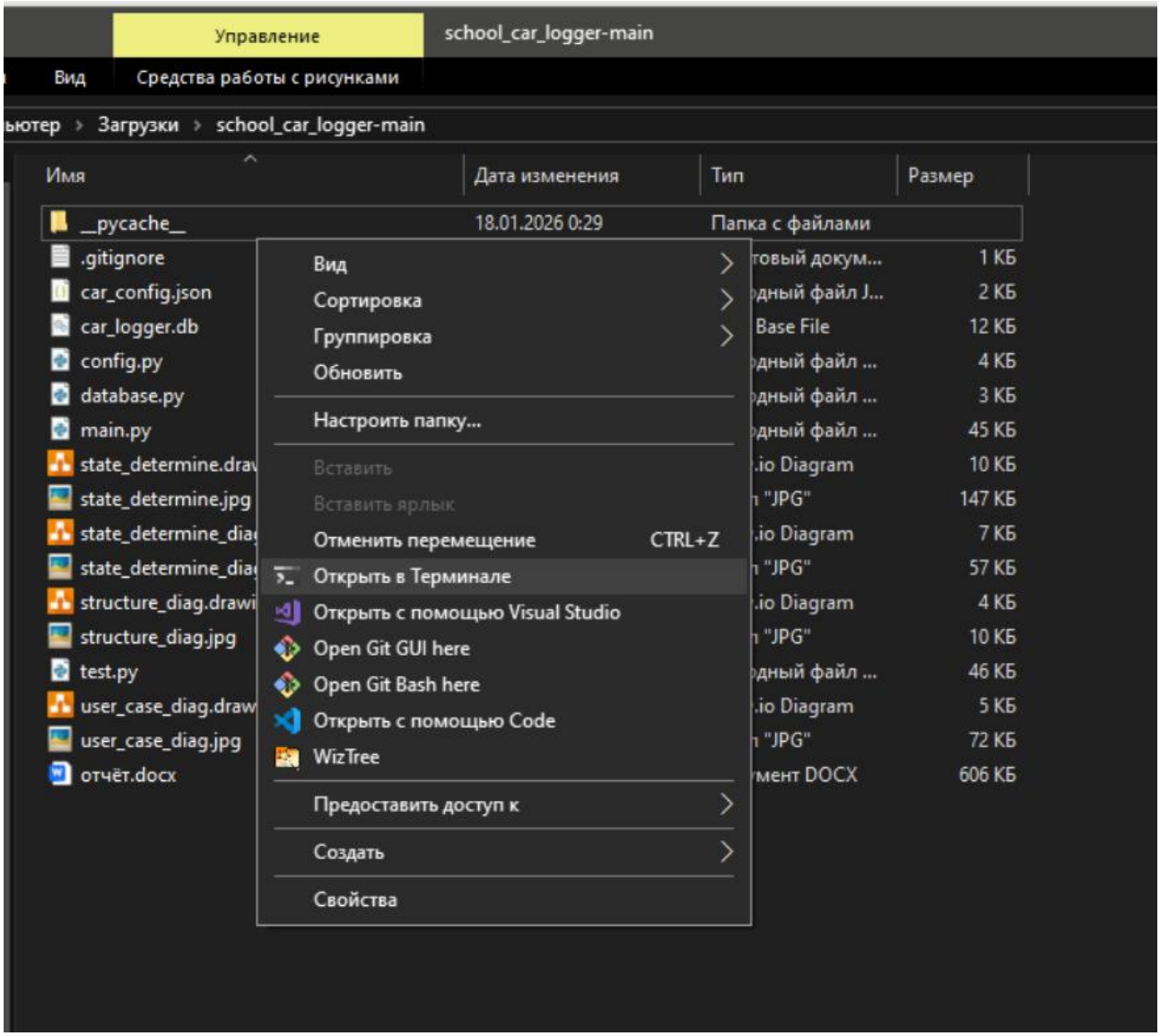


Рисунок 12 - Контекстное меню для открытия Терминала

4. Установим зависимости. Для этого в командной строке необходимо выполнить команду «pip install tk»;

5. Запустим приложение - «python.exe .\main.py».